

CALCULATE and FILTER CONTEXT

1. What is CALCULATE?

- CALCULATE is the single most important function in all of DAX.
- Every advanced DAX formula - time intelligence, running totals, percentage of total, ranking, dynamic segmentation - either uses CALCULATE directly or depends on a function that calls it internally.

One-sentence definition

CALCULATE evaluates an expression inside a modified filter context.

- You already know what filter context is from Module 5 - the set of rows visible to a measure at the time it evaluates.
- CALCULATE gives you programmatic control over that context: add filters, remove filters, replace filters, or override them entirely - before your expression runs.

1.1 Syntax

```
CALCULATE( <Expression> [, <Filter1>] [, <Filter2>] ... )
```

Argument	Description
<Expression>	The measure or expression to evaluate. This is WHAT you are calculating - SUM, COUNTROWS, DIVIDE, or any DAX expression.
<Filter1>, <Filter2>...	Optional. Each filter argument modifies the filter context before the expression evaluates. Multiple arguments act as AND conditions - all must be satisfied simultaneously.

1.2 CALCULATE EXECUTION SEQUENCE

When Power BI evaluates a CALCULATE formula, it follows this sequence every time without exception:

1. Read the existing filter context - all active filters from slicers, visual axes, page filters, and report filters
2. Process each filter argument - column conditions add or replace filters on specific columns; ALL() and REMOVEFILTERS() remove filters; FILTER() applies row-level conditions
3. Evaluate the expression inside the new modified filter context
4. Return the result - the modified context is discarded after evaluation

Critical misconception to avoid

- CALCULATE does not iterate rows.
- It modifies the filter context as a whole before the expression runs.
- Row iteration is FILTER's job. Describing CALCULATE's sequence as 'evaluating each row' is incorrect and will cost marks in interviews.

2. CALCULATE Categories

- Every CALCULATE formula falls into one of four categories.
- Understanding all four is essential before writing any advanced DAX.

2.1 Add a filter

- CALCULATE adds a new filter condition to the existing filter context.
- If no filter previously existed on that column, the new filter is simply applied.

Example

```
West Sales =  
CALCULATE (  
    SUM(Orders[Sales]),  
    People[Region] = "West"  
)
```

Working:

- People[Region] = "West" - adds a Region filter to the context
- If no Region slicer is active, this filter is applied fresh
- SUM(Orders[Sales]) evaluates only on rows where Region = West

2.2 Override an existing filter

- When a filter argument conflicts with an existing slicer or visual filter on the same column, CALCULATE replaces it - not adds to it.
- The slicer's filter is overwritten before the expression evaluates.

```
West Sales =  
CALCULATE (  
    SUM(Orders[Sales]),  
    People[Region] = "West"  
)
```

Working:

- If a Region slicer is set to East:
- CALCULATE replaces East with West
- SUM sees only West rows regardless of slicer

Why override matters

- This is the mechanism that makes hardcoded benchmark measures possible - West Sales always shows West regardless of what any slicer says.

- The slicer's filter context reaches CALCULATE, CALCULATE replaces it with its own filter, and the expression never sees the slicer's condition.

2.3 Remove a filter using ALL or REMOVEFILTERS

- CALCULATE can remove existing filters from the context entirely.
- Used for grand total denominators, benchmark calculations, and percentage-of-total measures.

```
Grand Total Sales =  
CALCULATE (  
    SUM(Orders[Sales]),  
    ALL(Orders)  
)
```

Working:

- ALL(Orders) removes every filter from the Orders table
- SUM evaluates on all 9,994 rows unconditionally
- Result: \$2,297,201 regardless of any slicer

2.4 Combine multiple filters (AND logic)

- Multiple filter arguments inside CALCULATE are AND conditions - every condition must be satisfied simultaneously.
- Each additional filter further narrows the rows visible to the expression.

```
West Technology Sales =  
CALCULATE (  
    SUM(Orders[Sales]),  
    People[Region] = "West",  
    'Product'[Category] = "Technology"
```

```
)
```

Working:

- Both conditions must be true on each row
- Orders filtered to: Region = West AND Category = Technology
- SUM evaluates on only those qualifying rows

3. FILTER REMOVAL FUNCTIONS

- These three functions are used exclusively inside CALCULATE to remove filters with varying degrees of precision.
- Choosing the wrong one is the most common source of incorrect percentage measures.

3.1 ALL

- ALL() removes the all existing filters across the given table or column

Syntax

```
ALL ( <TableOrColumn> )
```

Arguments

- <TableOrColumn> - Either a full table name (removes all filters from every column) or a Table[Column] reference (removes filter on that specific column only). When used inside CALCULATE, removes the specified filters from the context.

Example

```
% of Grand Total =  
DIVIDE (  
    SUM(Orders[Sales]) ,  
    CALCULATE (SUM(Orders[Sales]) , ALL(Orders)) ,  
    0
```

)

3.2 ALLEXCEPT()

- ALLEXCEPT removes all filters from a table EXCEPT the columns you explicitly name.
- It is the most precise filter removal tool when multiple filters exist and only some should be removed.

Syntax

```
ALLEXCEPT ( <Table>, <Column1> [, <Column2>] ... )
```

Argument	Description
<Table>	The table to remove filters from. Required - always a table, never a column.
<Column1>...	The columns whose filters are KEPT. All other column filters on the table are removed. Must reference columns from the same table.

Example

```
% Within Region (date-aware) =  
DIVIDE (  
    SUM(Orders[Sales]),  
    CALCULATE (  
        SUM(Orders[Sales]),  
        ALLEXCEPT(Orders, Orders[Order Date])  
    ),  
    0  
)
```

Working:

- ALLEXCEPT removes all filters from Orders EXCEPT Order Date
- Region filter is removed (giving region-independent total as denominator)

- Date filter on Orders remains active (denominator respects date slicer)
- Category filter removed

Common ALLEXCEPT mistake

- ALLEXCEPT only removes filters from the specified table.
- Filters on related dimension tables (People, Product, Date) are separate and are not affected by ALLEXCEPT(Orders, ...).
- If a Date slicer filters the Date dimension table and you use ALLEXCEPT(Orders, Orders[Order Date]), the Date dimension filter still flows through the relationship - you do not need to name Order Date to preserve it.
- Name only columns from the table specified in the first argument.

3.3 REMOVEFILTERS()

- Removes all filters in the table

Syntax

```
REMOVEFILTERS ( [<TableOrColumn>] )
```

Arguments

- [<TableOrColumn>] - Optional. A table or column to remove filters from. If omitted, removes all filters from all tables. Unlike ALL(), REMOVEFILTERS only works inside CALCULATE - it cannot return a table for use in other functions.

Example

```
% of Grand Total (modern) =  
DIVIDE (  
    SUM(Orders[Sales]),  
    CALCULATE (SUM(Orders[Sales]), REMOVEFILTERS (Orders)) ,
```

```
0
)
```

Working:

- Removes the filters in Orders Table
- Then calculates the SUM

Reference Table:

Function	What it removes	When to use
ALL (Table)	All filters on every column of the table	Grand total denominator - ignore everything
ALL (Table [Column])	Filter on one specific column only	Remove one column's filter, keep all others
ALLEXCEPT (Table, Col1, Col2)	All filters EXCEPT the named columns	Keep specific column filters, remove the rest
REMOVEFILTERS (Table)	All filters - dedicated filter modifier only	Modern preferred alternative to ALL(Table)

3.1 ALL(Table) vs ALL(Table[Column])

```
- Remove ALL filters from Orders (grand total denominator)
% of Grand Total =
DIVIDE (
    SUM(Orders[Sales]),
    CALCULATE (SUM(Orders[Sales]), ALL(Orders)),
    0
)

- Remove ONLY Category filter (keep Region filter active)
% Within Region =
DIVIDE (
```



```

SUM(Orders[Sales]) ,
CALCULATE(SUM(Orders[Sales]) , ALL('Product'[Category])) ,
0
)

```

Working:

- ALL(Orders) - Region filter removed, Category filter removed, Date filter removed. Denominator is always grand total.
- ALL('Product'[Category]) - only Category filter removed.
- Region filter stays active. In a matrix with Region on rows, denominator is the current region's total. Each region's categories sum to 100% independently.

3.3 REMOVEFILTERS vs ALL

Property	ALL(Table)	REMOVEFILTERS(Table)
Result as CALCULATE modifier	Identical - both remove all filters from the table	Identical - both remove all filters from the table
Can return a table?	Yes - ALL() is a table function usable in FILTER, SUMX etc.	No - only works inside CALCULATE as a filter modifier
Affects relationships?	No - relationships are never modified by any DAX function	No - relationships are never modified by any DAX function
Microsoft recommendation	Legacy - still valid but dual-purpose creates ambiguity	Preferred in modern DAX - intent is immediately clear

Critical correction - ALL does NOT remove relationships

- ALL() removes filter context only.
- Relationships in Power BI are part of the data model and are never modified, overridden, or removed by any DAX function at query time.
- This is a common misconception that must not appear in interviews or production code documentation.

4. FILTER IN CALCULATE

- Simple column filter arguments inside CALCULATE apply conditions as independent table filters.
- When two conditions must both be true on the same row simultaneously, FILTER with && is required.

4.1 Simple column filter vs FILTER

Property	Column filter arguments	FILTER with &&
Evaluation	Applies each condition as a separate independent table filter	Iterates rows one at a time - both conditions checked on the same row
Row-level co-occurrence	Not guaranteed - conditions applied independently	Guaranteed - && ensures both conditions true on the same row
Performance	Faster - column filters are highly optimised internally	Slower - row iteration is more expensive than column filters
When to use	When conditions on separate columns are independent	When two or more conditions must be true on the same row

- Simple column filter arguments (independent evaluation)

Loss Orders Simple =

```
CALCULATE(  
    COUNTROWS(Orders),  
    People[Region] = "West",  
    Orders[Profit] < 0  
)
```

- FILTER with && (row-level co-occurrence guaranteed)

Loss Orders Filter =

```
CALCULATE(  
    COUNTROWS(Orders),  
    FILTER(Orders, Orders[Profit] < 0 && People[Region] = "West")
```

```

    FILTER(
        Orders,
        RELATED(People[Region]) = "West" && Orders[Profit] < 0
    )
)

```

RELATED inside FILTER - mandatory for dimension columns

- When FILTER iterates a fact table (Orders), references to dimension table columns (People[Region], Product[Category]) require RELATED() to traverse the relationship.
- Writing People[Region] directly inside FILTER on Orders causes an error or wrong results. RELATED(People[Region]) correctly fetches the Region value for each Orders row via the relationship.

5. Context Transition

- Context transition is the mechanism by which CALCULATE converts an active row context into an equivalent filter context.
- It is the most advanced concept in Module 6 and the reason CALCULATE is the only bridge between row context and filter context in DAX.

5.1 The problem and what context transition solves

From Module 5: aggregation functions (SUM, COUNT, AVERAGE) ignore row context - they scan the entire column unconditionally. This means:

```

- Calculated column on Orders table
Wrong Column = SUM(Orders[Sales])

```

Working:

- Returns \$2,297,201 on every row
- SUM has no awareness of which customer is on the current row

Context transition via CALCULATE is the solution - the only way to make an aggregation function aware of the current row.

5.2 How context transition works mechanically

When CALCULATE executes inside an active row context, it automatically reads every column value of the current row and converts them into filter conditions:

Example:

Current row values (row context)	Filter conditions CALCULATE creates
Customer ID = CG-12520	Orders[Customer ID] = "CG-12520"
Region = West	Orders[Region] = "West"
Sub-Category = Chairs	Orders[Sub-Category] = "Chairs"
Order Date = 2016-01-03	Orders[Order Date] = "2016-01-03"
...all 12 columns	...12 simultaneous filter conditions

- CALCULATE creates these filter conditions automatically - you do not write them manually.
- This is a context transition.

5.3 Context transition on the same table

When CALCULATE with no filter arguments is used inside a calculated column on the same table it references, context transition fires and creates filters on ALL columns of the current row simultaneously. With 12 columns, 12 filters are applied - uniquely identifying the single current row. The result is effectively just that row's own column value.

```
- Calculated column on Orders table  
Test Column = CALCULATE(SUM(Orders[Sales]))
```

Working:

- Context transition creates 12 simultaneous filters
- These 12 filters uniquely identify the current row
- SUM returns only that row's Sales value
- Equivalent to just writing: Orders[Sales]

Context transition on the same table with no filter arguments is rarely useful

- The 12 simultaneous filters uniquely identify the current row - making the result identical to reading the column directly.
- Context transition produces meaningful grouped results only when (a) used on a related dimension table via RELATEDTABLE, or (b) combined with ALL() and EARLIER() to explicitly control which column the filter applies to.

5.4 Context transition on a related table - the useful case

- Context transition produces genuine value when CALCULATE is used in a calculated column on a dimension table referencing a related fact table.
- The current dimension row's context filters the fact table through the relationship.

```
- Calculated column on the Customers dimension table  
Customer Total Sales =  
CALCULATE(  
    SUM(Orders[Sales]),  
    RELATEDTABLE(Orders)  
)  
- Without CALCULATE:
```

Wrong = SUM(Orders[Sales]) - returns \$2,297,201 on every row

Working:

- For Customer ID = CG-12520:
- RELATEDTABLE(Orders) returns all Orders rows where Customer ID = CG-12520
- CALCULATE uses this as the filter context
- SUM evaluates only on that customer's orders
- Result: total sales for Customer CG-12520 only

5.5 Context transition inside iterators - SUMX

Context transition also fires when CALCULATE is placed inside an iterator function (SUMX, AVERAGEX, FILTER). The iterator creates row context for each iteration - CALCULATE converts that row context into filter context automatically.

Correct Total = SUMX(Orders, Orders[Sales])

CALCULATE needed when aggregating a related table

Customer Revenue =

```
SUMX (  
    Customers,  
    CALCULATE (SUM(Orders[Sales]), RELATEDTABLE(Orders))  
)
```

Working:

- Orders[Sales] reads each row's value using row context
- SUMX sums all row values = \$2,297,201
- SUMX iterates Customers row by row
- CALCULATE triggers context transition for each customer
- RELATEDTABLE filters Orders to that customer's rows
- SUM aggregates only those rows per iteration

Rule for SUMX with CALCULATE

- Use SUMX(Table, Table[Column]) when you simply need to sum column values row by row - no CALCULATE needed.
- Use SUMX with CALCULATE only when the row-level expression requires aggregating data from a related table or applying additional filter logic that cannot be expressed as a simple column reference.

6. CALCULATING PERCENTAGES

Percentage measures are the most common real-world application of CALCULATE with filter removal. Three standard patterns cover almost every business requirement.

6.1 % of Grand Total

```
% of Grand Total =  
DIVIDE(  
    SUM(Orders[Sales]),  
    CALCULATE(SUM(Orders[Sales]), ALL(Orders)),  
    0  
)
```

Working:

- Numerator: current filter context (respects all slicers)
- Denominator: ALL(Orders) removes every filter - always \$2,297,201
- Result: Technology = 36.4%, Furniture = 32.3%, Office Supplies = 31.3%
- All percentages across entire report sum to 100%

6.2 % Within Region

```
% Within Region =  
DIVIDE(  
    SUM(Orders[Sales]),
```

```

    CALCULATE (SUM(Orders[Sales]), ALL('Product'[Category])),
    0
)

```

Working:

- Numerator: current filter context
- Denominator: Category filter removed, Region filter kept
- In a matrix with Region rows: denominator = current region total
- Each region's categories sum to 100% independently
- West: Technology 34.7% + Furniture 31.3% + Office Supplies 34.0% = 100%

6.3 % of Profitable Orders (filter + percentage combined)

```

Profitable Sales % =
DIVIDE(
    CALCULATE(
        SUM(Orders[Sales]),
        FILTER(Orders, Orders[Profit] > 0)
    ),
    SUM(Orders[Sales]),
    0
)

```

Working:

- Numerator: CALCULATE filters to profitable rows only
- Denominator: SUM with no modification - all rows in filter context
- Result: % of total sales that came from profitable orders
- Respects all report slicers in both numerator and denominator

7. Mistakes Made and Lessons Learned

Mistake 1: Described CALCULATE's sequence as row iteration

What happened: Described the execution sequence of CALCULATE as iterating through rows.

Why it was wrong: CALCULATE does not iterate rows; it modifies the filter context as a whole. Row iteration is the specific job of the FILTER function.

Fix: Correctly describe the sequence: 1) Read existing filter context, 2) Process each filter argument, 3) Evaluate expression in the modified context, 4) Return result.

Lesson: CALCULATE modifies filter context as a whole and does not loop through rows.

Mistake 2: Used People[Region] inside FILTER on Orders without RELATED

What happened: Directly referenced a dimension table column inside a FILTER function iterating over a fact table.

Why it was wrong: When iterating a fact table, you cannot directly reference dimension columns; you must traverse the relationship to fetch the data.

Fix: Always use the RELATED function when referencing dimension columns while iterating over a fact table: `FILTER(Orders, RELATED(People[Region]) = "West")`.

Lesson: Use `RELATED(DimTable[Column])` when referencing dimension columns inside FILTER on a fact table.

Mistake 3: Wrong table for ALL - wrote Orders[Category] instead of Product[Category]

What happened: Attempted to use a column from a different table in the ALL function.

Why it was wrong: The column [Category] does not exist in the Orders table, which causes a syntax error.

Fix: Reference the table where the column actually resides: `ALL('Product'[Category])`.

Lesson: Always reference the table where the column is defined.

Mistake 4: Explained why West Sales ignores East slicer incorrectly

What happened: Claimed that CALCULATE results in zero because there are no East rows in the filtered dataset.

Why it was wrong: CALCULATE replaces the existing filter context; it does not simply result in zero. The slicer's filter is overwritten before evaluation.

Fix: Explain that CALCULATE replaces the existing Region filter with its own specified filter, overwriting the slicer's condition.

Lesson: CALCULATE replaces existing filters on the same column; it does not just filter out non-matching rows.

Mistake 5: Claimed ALL removes relationships

What happened: Incorrectly stated that the ALL function removes relationships from the data model.

Why it was wrong: Relationships are permanent model structures; ALL only removes filter context and does not impact model relationships at query time.

Fix: Clarify that ALL removes filters only, while relationships remain intact.

Lesson: Relationships are permanent model-level structures and are never modified by DAX functions at runtime.

Mistake 6: Used ALLEXCEPT('People'[Region]) - column as first argument

What happened: Used a column as the first argument in ALLEXCEPT.

Why it was wrong: ALLEXCEPT requires a table name as its first argument, not a column. This causes a syntax error.

Fix: Use the table name as the first argument: ALLEXCEPT(Orders, Orders[Order Date]).

Lesson: ALLEXCEPT requires a table as the first argument.

Mistake 7: Used CALCULATE(SUM(...)) with no filter args as fix for SUMX

What happened: Used CALCULATE with no filter arguments inside a SUMX iterator.

Why it was wrong: This triggers context transition for all columns, which is unnecessarily complex when a direct column reference would suffice.

Fix: Use a simple column reference: SUMX(Orders, Orders[Sales]). Use CALCULATE inside SUMX only when aggregating a related table.

Lesson: Direct column references are cleaner than CALCULATE inside SUMX unless related table aggregation is required.

8. DAX REFERENCE

8.1 RELATEDTABLE

- RELATEDTABLE is called from the one side and returns all rows from the many side that match the current row's key.
- Always returns a table - use inside CALCULATE or aggregation functions, never standalone.

Syntax

```
RELATEDTABLE ( <TableName> )
```

Arguments

- <TableName> - A related table on the many side of a relationship

Example

```
Customer Total Sales =
```

```

CALCULATE (
    SUM(Orders[Sales]) ,
    RELATEDTABLE (Orders)
)

```

Working:

- RELATEDTABLE returns all Orders rows related to the current customer via Customer ID. CALCULATE uses this as the filter context
- SUM evaluates only on that customer's orders.

8.2 ALLSELECTED

- ALLSELECTED removes filters applied by slicers while keeping filters that come from the visual structure (rows, columns, axes).
- This gives the currently-selected subset as context.

Syntax

```

ALLSELECTED ( [<TableOrColumn>] )

```

Arguments

- [<TableOrColumn>] - Optional. A table or column.

Example

```

% of Selected Total =
DIVIDE (
    SUM(Orders[Sales]) ,
    CALCULATE (SUM(Orders[Sales]) , ALLSELECTED (Orders)) ,
    0
)

```

When a Region slicer is set to West:

Working:

- ALLSELECTED keeps the West context as the denominator - percentages sum to 100% within the slicer selection.
- Then calculates Numerator - SUM

ALL vs ALLSELECTED

- ALLSELECTED removes filters, it never adds them.
- Never use ALLSELECTED as a fix for context transition problems.